VIVA PREPARATION

Viva Question Bank

Questions an examiner is likely to ask about the METR-LA traffic forecasting project, with short model answers you can say aloud.

How to use this

Read the key-numbers sheet first, then each section. Every answer is two to five sentences: say the first sentence, then add detail if asked.

core almost certain to be asked deeper follow-up or theory tricky designed to test understanding

Numbers come from `results/tables/` and `results/metrics/`. If you re-run any experiment, re-check them.

Key numbers to remember

207 × 34,272
sensors × 5-min steps (1 Mar – 27 Jun 2012)

8.1%
missing readings (stored as 0); 12.1% in test vs 7.1% in train

23,974 / 3,425 / 6,850
train / val / test windows (70/10/20 by time)

1,722
graph non-zeros (1,515 edges + self-loops), 73% one-way

Test MAE in mph (lower is better).

Model	15 min	30 min	60 min	Avg	RMSE avg	MAPE avg
Historical Average	4.19	4.19	4.19	4.19	7.85	13.0%
Persistence	3.50	4.21	5.38	4.24	8.19	11.0%
Ridge (34 features)	3.37	3.78	4.09	3.69	6.46	10.3%
Gradient Boosting (34 features)	2.89	3.29	3.66	3.22	6.43	9.2%
Graph WaveNet	**2.72**	**3.10**	**3.59**	**3.07**	**6.21**	**8.4%**
Graph WaveNet, published (Wu et al. 2019)	2.69	3.07	3.53	–		
DCRNN, published (Li et al. 2018)	2.77	3.15	3.60	–		

Other facts	Value
Speed mean ± std (median)	58.5 ± 13.0 (63.2) mph
ACF at 5 min / 1 h / 1 day / 1 week	0.86 / 0.57 / 0.36 / 0.40
Anomaly correlation: neighbours vs not	0.27 vs 0.07
Graph WaveNet parameters	300,952
GWN best epoch / val MAE	25 / 2.81 mph
GWN gain over gradient boosting	4.5% (avg MAE)

More facts	Value
DM stat GWN vs GBM (15/30/60)	−25.7 / −15.1 / −3.6
95% CI avg MAE, GWN	2.89 – 3.20
60-min MAE, latest input missing	GWN 5.36, GBM 4.50
Top feature (both horizons)	profile_target
Features / groups	34 / 6
Unit tests	23

A. The project in general

Q1 **Explain your project in two minutes.** core

We forecast freeway traffic speed at 207 loop detectors in Los Angeles, 5 to 60 minutes ahead, from the last hour of readings. We use the METR-LA benchmark and compare a ladder of six models: Historical Average, Persistence, Ridge and gradient boosting on 34 hand-engineered features, an LSTM, and Graph WaveNet, a graph neural network that uses the road network.

Graph WaveNet is best, with an average error of 3.07 mph; the improvement over gradient boosting is statistically significant at every horizon. The project also covers a data-quality analysis, leakage-tested features, significance tests, error analysis, unit tests and an interactive web app.

Q2 **What exactly is the input and the output?** core

Input: a $12 \times 207 \times 2$ tensor, the last 12 five-minute steps for all 207 sensors, with two channels (normalised speed and time of day), plus the road graph. Output: a 12×207 matrix, the speed at every sensor for each of the next 12 steps (5 to 60 minutes). It is multi-step, multi-output forecasting.

Q3 **Why is this a machine-learning problem and not just "use the average"?** core

The average is a fair baseline (4.19 mph), but traffic often deviates from its usual pattern: incidents, weather, demand spikes. These deviations persist for tens of minutes and spread along roads. Learning to forecast the *deviation* is where models add value: Graph WaveNet cuts the error by 27% relative to the average.

Q4 **Who would use these forecasts?** core

Traffic management centres (ramp metering, variable message signs, incident response), navigation apps (route guidance and arrival-time estimates) and logistics planners. A 15–60 minute horizon gives enough time to act.

Q5 **What is new or your own in this project, given that Graph WaveNet already exists?** tricky

We did not invent the model. Our contribution is a careful, fair comparison: (1) a data-quality and EDA study that links each finding to a design decision; (2) a strong, leakage-tested feature set for classical models, measured by ablation and permutation importance; (3) a compact re-implementation of Graph WaveNet that reproduces published numbers within 0.03–0.06 mph; (4) significance testing (Diebold–Mariano, block bootstrap), which most papers skip; (5) error analysis by horizon, hour, sensor and missing input; and (6) an interactive app.

B. Dataset and exploratory analysis

Q6 **Describe METR-LA.** core

207 loop detectors on Los Angeles County freeways, 5-minute average speeds from 1 March to 27 June 2012: 34,272 time steps, 119 days. Speeds are in mph, capped at 70. It comes with sensor coordinates and pairwise road-network distances. It was released with DCRNN (Li et al., ICLR 2018) and is the standard benchmark for spatio-temporal forecasting.

Q7 **What is a loop detector?** core

An inductive wire loop cut into the road surface. A car passing over it changes the loop's inductance, so the detector counts vehicles and their occupancy time. Speed is estimated from these, and here it is averaged over 5 minutes.

Q8 **How are missing values stored, and how much is missing?** core

As 0 mph, which is impossible on a freeway in practice, so it means "sensor reported nothing". 8.1% of all readings; per sensor 6.3–20.1%. There are network-wide outages (e.g. 26–27 March, when more than 50% is missing) and long single-sensor failures (the median sensor's longest gap is 381 steps, about 32 hours).

Q9 **How did you handle the missing values?** core

Four ways. (1) Missing *targets* are masked out of every loss and metric. (2) For tabular features we use causal forward-fill (last observation carried forward). (3) Deep models see the raw 0 in the input, as in the standard pipeline; after z-scoring it becomes -4.57, a distinctive value the network can learn to recognise. (4) We analyse errors separately for windows whose latest input is missing.

Q10 **Why not interpolate the gaps?** tricky

Linear interpolation uses the next observed value, which lies in the future at prediction time. That is leakage. Forward-fill uses only the past. We also never impute *labels*: a model should not be scored on values we invented.

Q11 **What did the EDA tell you, and what did each finding change?** core

Lag-1 autocorrelation of 0.86 → lag features. A weekly cycle (1-week ACF 0.40 is above 1-day 0.36) → weekly-profile and calendar features. The ADF test rejects a unit root → no differencing needed. Neighbours on the graph stay correlated after de-seasonalising (0.27 vs 0.07) → spatial features and a GNN. The left-skewed distribution (skew -1.99) → MAE loss. The test period has more missing data → a robustness analysis.

Q12 Is the series stationary? deeper

The Augmented Dickey–Fuller test rejects a unit root for all ten sampled sensors ($p < 10^{-24}$). Speed is bounded and mean-reverting. The remaining non-stationarity is *seasonal*, a mean that depends on time of week, and we model it explicitly with the weekly profile instead of differencing.

Q13 What is the "anomaly" you keep mentioning? deeper

Current speed minus the sensor's usual (training-period) speed at this time-of-week slot. Removing the weekly profile drops the 1-day autocorrelation from 0.36 to 0.07, so the anomaly is a short-memory process (ACF 0.41 at 1 hour). Forecasting becomes "usual level + decaying anomaly".

Q14 Why are speeds capped at 70 mph and does it matter? tricky

The sensors' processing clips values at 70. Only 0.5% of readings sit exactly at the cap, so it barely affects the metrics. It does make the distribution bounded on the right, which adds to the left skew.

C. Splitting, scaling and leakage

Q15 How did you split the data, and why not random K-fold? core

Chronologically 70/10/20: train to 23 May, validation to 4 June, test to 27 June (23,974 / 3,425 / 6,850 windows). Adjacent windows share 11 of 12 input steps, so a random split would put near-copies of test samples in training and inflate scores. In deployment we always predict the future from the past. The split matches DCRNN and Graph WaveNet, so numbers are comparable.

Q16 How do you know there is no leakage? core

(a) A chronological split. (b) The scaler, weekly profile and sensor statistics are fitted on the training period only, with unit tests `test_scaler_uses_train_only` and `test_profile_uses_training_period_only`. (c) Every feature uses data at or before t: `test_no_future_leakage` overwrites all values after the anchor and asserts the features are unchanged. (d) Hyper-parameters are chosen on validation; the test set is used once.

Q17 Calendar features use the target time t + h. Isn't that leakage? tricky

No. The clock time of the target is known in advance: at 16:30 we know the 60-minute forecast is for 17:30. Only target *values* would be leakage.

Q18 How are inputs normalised for the deep models? deeper

Z-score with mean 58.58 and std 12.83 mph, computed on *observed* (non-zero) training values only. Time of day is a fraction in [0, 1). The model predicts in normalised units, and the trainer converts back to mph before computing the loss, so the loss is in mph.

Q19 Why every 4th training window for the tabular models? deeper

Consecutive windows overlap in 11 of 12 steps, so they are highly redundant. Every 4th still gives about 1.15 million labelled rows per horizon (×207 sensors), at a quarter of the memory and time, with little loss of information.

Q20 Why is validation split into even and odd windows? deeper

Even windows select hyper-parameters; odd windows drive gradient boosting's early stopping. Using the same rows for both would overfit the selection to the early-stopping set.

D. Feature engineering

Q21 List your features and why each group exists. core

34 features in 6 groups. **Lags (12)**: speed at t … t−11, because of strong autocorrelation. **Window stats (7)**: 1- and 3-step differences, rolling mean/std/min, number missing, missing-now flag, for trend, volatility and data quality. **Calendar (4)**: sin/cos time of day of the target, day of week, weekend, for cycles. **Weekly profile (3)**: usual speed at the target slot, usual speed now, and the anomaly. **Spatial (5)**: mean speed and anomaly of downstream and upstream neighbours, plus the downstream trend, because jams travel upstream. **Sensor (3)**: each sensor's training mean, std and missing rate, as an identity signal.

Q22 How do you know the features help? core

Two ways. A cumulative ablation adds one group at a time and measures validation MAE: at 60 minutes it falls from 4.36 (lags only) to 3.96 with calendar, 3.52 with the weekly profile, 3.43 with spatial and 3.41 with sensor stats. Permutation importance on held-out rows shows which individual columns matter.

Q23 Which feature matters most, and why does importance change with horizon? core

`profile_target`, the usual speed at the target's time-of-week, is top at both horizons (shuffling it adds 5.3 mph at 15 min and 6.8 mph at 60 min). Recent-state features matter far more at short horizons: `lag_0` adds 2.4 mph at 15 min but only 0.5 at 60 min. The current anomaly decays, so long forecasts revert towards the seasonal mean.

Q24 Why permutation importance rather than the trees' built-in importance? deeper

Impurity (split-gain) importance is biased towards continuous, high-cardinality features and is measured on training data. Permutation importance measures the real increase in held-out MAE when one column is shuffled. One caveat: it splits credit among correlated features (`lag_0`, `lag_1` and `roll_mean` are near-duplicates), which is why we also run the group ablation.

Q25 Why sin/cos for time of day? core

A raw 0–287 index puts 23:55 and 00:00 at opposite ends. Mapping it to a circle with sin and cos keeps them close, so even a linear model can use it smoothly.

Q26 Why are downstream neighbours an "early warning"? deeper

A queue forms at a bottleneck and grows backwards, against the direction of travel. So the sensor downstream of you slows first, and the slowdown reaches you minutes later. The forward random-walk matrix P_f aggregates exactly those downstream sensors.

Q27 Why does the weekend flag have zero importance? tricky

Its information is already in `dow` (day of week) and, more strongly, in `profile_target`, which knows the usual weekend speed for every slot. Permutation importance gives zero to features that are fully redundant.

E. Models and theory

Q28 Why this particular set of six models? core

It is a ladder of increasing complexity, so each step's gain can be attributed: seasonality only (HA) → recency only (Persistence) → linear use of features (Ridge) → non-linear use of features (gradient boosting) → learned temporal features with no space (LSTM) → learned spatio-temporal features (Graph WaveNet).

Q29 Write down the training objective. core

$$\mathcal{L} = \frac{\sum_{h,i} m_{h,i} \, |\hat{y}_{h,i} - y_{h,i}|}{\sum_{h,i} m_{h,i}}, \quad m = 1[y > 0]$$

The masked mean absolute error over all 12 horizons and 207 sensors, ignoring missing targets.

Q30 Why MAE and not MSE as the loss? core

MAE's minimiser is the conditional median, which is robust to the heavy left tail of sudden congestion; MSE would let rare huge errors dominate. It also matches the headline metric. The trade-off is that RMSE is not optimised directly.

Q31 Why does Persistence beat the Historical Average at 15 minutes but lose at 60? core

Persistence is optimal for a random walk. Traffic is mean-reverting, so "same as now" is excellent a few minutes ahead but drifts further from the truth as the horizon grows (3.50 → 5.38 mph). HA ignores the present, so its error is flat (4.19). The crossover is around 30 minutes, and the learned models combine the two.

Q32 **Explain Ridge regression and why you needed the penalty.** deeper

$\hat{\beta} = (X^\top X + \lambda I)^{-1} X^\top y$. The lags are highly collinear (ACF 0.86), so $X^\top X$ is ill-conditioned and plain least squares would have unstable coefficients. The L2 penalty shrinks low-variance directions, trading a little bias for less variance. λ was picked on validation (1000), but validation MAE was almost flat across λ, so the model is not sensitive to it.

Q33 **How does gradient boosting work?** core

It builds an additive model $F_M(x) = F_0 + \sum_m \nu\, h_m(x)$. Each small tree h_m fits the negative gradient of the loss at the current prediction. For absolute error that gradient is sign(y − F), and the leaf values are medians of the residuals, which makes it robust. The learning rate ν shrinks each step. We used scikit-learn's *histogram* version: features are binned into at most 255 bins, so finding a split costs O(bins), not O(n log n).

Q34 **Why one model per horizon (direct) instead of feeding predictions back (recursive)?** deeper

The recursive strategy feeds the model's own 1-step forecasts back in, so errors compound over 12 steps. The direct strategy trains a separate model per horizon (tabular) or one model with 12 outputs (deep), which avoids accumulation and lets each horizon weight features differently. The cost is more models (Ben Taieb et al., 2012).

Q35 **Write the LSTM equations and say why it suffers less from vanishing gradients.** deeper

$i, f, o = \sigma(W[h_{t-1}, x_t] + b)$, $g = \tanh(W_g[h_{t-1}, x_t] + b_g)$, $c_t = f \odot c_{t-1} + i \odot g$, $h_t = o \odot \tanh(c_t)$. The cell state is updated *additively*, so gradients can flow through many steps without repeated squashing. Our LSTM is shared across sensors and sees no neighbours, so it isolates the value of temporal learning alone.

Q36 **How is the adjacency matrix built?** core

$W_{ij} = \exp(-d_{ij}^2/\sigma^2)$ if at least 0.1, else 0, where d_{ij} is the *road* distance from i to j and σ the standard deviation of all distances. The Gaussian kernel turns distance into similarity, and the threshold keeps it sparse: 1,722 non-zeros, exactly as in DCRNN's preprocessing. It is asymmetric because road distance is directional (73% of edges are one-way).

Q37 **What is diffusion convolution?** deeper

It treats traffic as a random walk on the graph. $P_f = D_O^{-1} W$ moves information downstream and $P_b = D_I^{-1} W^\top$ upstream. A K-step diffusion convolution sums $P^k X \Theta_k$ for k = 0…K in both directions, aggregating up to K hops. Graph WaveNet uses K = 2. With sparse matrices it costs O(K·|E|·C).

Q38 **What is the self-adaptive adjacency matrix?** core

$\tilde{A} = \mathrm{softmax}(\mathrm{ReLU}(E_1 E_2))$ with $E_1 \in \mathbb{R}^{207\times 10}$ and $E_2 \in \mathbb{R}^{10\times 207}$ learned end-to-end. ReLU removes weak links and the row softmax makes it a valid transition matrix. It can learn dependencies the distance graph misses, such as parallel freeways or sensors with no recorded edges (five have none outgoing). Its rank is at most 10, which regularises it.

Q39 **Explain the gated temporal convolution and the receptive field.** deeper

$h = \tanh(\Theta_1 \star x) \odot \sigma(\Theta_2 \star x)$. The sigmoid gate controls information flow like an LSTM gate, but it is convolutional, so all steps run in parallel. The convolutions are *causal* (no future inputs) and *dilated* (1, 2), with kernel 2 over 4 blocks. The receptive field is $1 + 4\cdot(1 + 2) = 13 \geq 12$, so every output sees the whole input hour; the input is left-padded by one step.

Q40 **Why a convolutional model instead of a recurrent one like DCRNN?** deeper

Convolutions process all time steps in parallel, so training is faster. There are no vanishing gradients through 12 recurrent steps and no exposure bias from feeding predictions back. Graph WaveNet also outperformed DCRNN in the literature (3.53 vs 3.60 mph at 60 minutes).

Q41 **What are the residual and skip connections for?** deeper

Residual connections add each layer's input to its output, so the layer only learns a correction; this makes deep stacks easier to train. Skip connections send every layer's output straight to the output head, so the head combines short-range (early layers) and long-range (later layers) temporal patterns.

Q42 **How many parameters does Graph WaveNet have, and where are most of them?** tricky

300,952. The largest single block is the first output convolution (256 → 512 channels): 131,584, or 44%. Then the eight skip convolutions (67,584), the eight diffusion convolutions (57,600; each mixes (3 supports × 2 orders + 1) × 32 input channels) and the gated filters (33,280). The adaptive embeddings are only 2 × 207 × 10 = 4,140. Because convolutions share weights over sensors and time, the count does not grow with the length of the series.

F. Training and tuning

Q43 **How did you train the deep models?** core

Adam with learning rate 10^{-3} and weight decay 10^{-4}, batch size 32, gradient-norm clipping at 5 and the masked MAE loss. After every epoch we compute validation MAE, save the best weights as a checkpoint, and stop early after 8 epochs without improvement. It ran on the Apple M3 GPU through PyTorch's MPS backend, at about 3 minutes per epoch.

Q44 How did you choose hyper-parameters? core

On the validation split only. Ridge: $\lambda \in \{0.01, 1, 100, 1000\}$. Gradient boosting: three configurations of learning rate × leaves, with early stopping (best: learning rate 0.1, 127 leaves). Graph WaveNet: the published architecture, with batch 32 instead of 64 to fit 8 GB of unified memory.

Q45 Your training curve shows training error above validation error. Isn't that strange? tricky

No. During training, dropout (0.3) is switched on and BatchNorm uses batch statistics, so the network is deliberately handicapped. Validation runs in evaluation mode without dropout. The training loss is also an average over the epoch while the weights are still improving. The validation curve flattening (2.81 at epoch 25) is the signal that matters, and we keep that checkpoint.

Q46 How long did Graph WaveNet train, and was it finished? tricky

Be honest here. It was planned for up to 35 epochs with patience 8. The run was interrupted after epoch 26 (epochs 18 and 19 also took 19 and 52 minutes instead of the usual 3, so the machine was paused or busy at that point). We use the best-validation checkpoint from epoch 25 (val MAE 2.81). Validation had nearly flattened, and the test numbers are within 0.03–0.06 mph of the published model, so a full run would change little. The paper trained for 100 epochs.

Q47 Why gradient clipping? deeper

It rescales the gradient when its norm exceeds 5, which prevents occasional huge updates, for example from batches with sudden congestion or many missing values, from destabilising training.

Q48 How do you avoid overfitting? core

Deep models: dropout 0.3, weight decay, early stopping on validation, and the best checkpoint. Gradient boosting: shrinkage, at least 100 samples per leaf, an L2 leaf penalty and early stopping. Ridge: the L2 penalty. All selection uses validation, and the test set is touched once.

Q49 Is the result reproducible? deeper

Seeds are fixed (NumPy and PyTorch seed 0), the split is deterministic, and every experiment is a script. GPU kernels (MPS) are not bit-for-bit deterministic, so deep results can vary by a few hundredths of a mph between runs; multi-seed runs are listed as future work.

G. Evaluation and statistics

Q50 Define MAE, RMSE and MAPE and say what each tells you. core

MAE is the average absolute error in mph: the typical miss. RMSE is the square root of the mean squared error, also in mph; it punishes large misses more, so it reflects behaviour during sudden congestion. MAPE is the average percentage error; it weighs errors at low speeds more. All three ignore missing targets.

Q51 **Why must MAPE exclude zero targets?** tricky

MAPE divides by the true value, so a 0 would give infinity. Here zeros are missing readings anyway, so the same mask removes them from all three metrics.

Q52 **How do you know the improvement is real and not noise?** core

Two methods. Diebold–Mariano tests on per-window absolute errors: Graph WaveNet vs gradient boosting gives -25.7, -15.1 and -3.6 at 15/30/60 min, all $p < 0.001$. And 95% day-level block-bootstrap confidence intervals: Graph WaveNet's average MAE is 2.89–3.20 mph.

Q53 **Explain the Diebold–Mariano test.** deeper

For each test window s, take the loss difference d_s between the two models (mean absolute error over sensors). Under the null of equal accuracy, $E[d_s] = 0$ and $DM = \bar{d} / \sqrt{\hat{\sigma}^2_{LR}/n}$ is approximately standard normal. Overlapping windows are autocorrelated, so the long-run variance uses a Newey–West estimator with Bartlett weights and h lags. A negative statistic means model A is more accurate.

Q54 **Why a *block* bootstrap and not an ordinary one?** deeper

Errors within a day are strongly dependent (a jam affects many consecutive windows). Resampling single windows would pretend they are independent and give intervals that are too narrow. Resampling whole days keeps the within-day dependence.

Q55 **The confidence intervals of Graph WaveNet and gradient boosting overlap. Doesn't that mean the difference isn't significant?** tricky

No. Overlapping marginal intervals do not imply an insignificant difference. Both models are evaluated on the same days, so their errors are highly correlated: a hard day is hard for both. The paired Diebold–Mariano test uses the *difference* per window, which cancels that shared variation, and it is clearly significant.

Q56 **Why report 15, 30 and 60 minutes specifically?** core

They are steps 3, 6 and 12, the standard horizons in the METR-LA literature (DCRNN, Graph WaveNet), so our numbers can be compared directly. We also report the average over all 12 steps.

H. Results and interpretation

Q57 **What are your headline results?** core

Graph WaveNet: 2.72 / 3.10 / 3.59 mph MAE at 15/30/60 minutes and 3.07 averaged. That beats gradient boosting (3.22), Ridge (3.69), Persistence (4.24) and the Historical Average (4.19). It is within 0.03–0.06 mph of the published Graph WaveNet and better than the published DCRNN at all three horizons.

Q58 Graph WaveNet is only 4.5% better than gradient boosting. Was the deep model worth it? tricky

It depends on the use case. The gain is largest at short horizons (6% at 15 minutes) and smallest at 60 minutes (2%), where the weekly profile dominates and both models revert towards it. Graph WaveNet needs no feature engineering, predicts all sensors and horizons in one pass (the full test set in about 15 s versus 84 s for gradient boosting) and is significantly better. But gradient boosting is easier to explain, cheaper to train on a CPU and more robust when inputs are missing. It is an honest trade-off, and a key finding of the project.

Q59 When are the models worst? core

In the evening rush (15:00–19:00), when 60-minute error peaks around 6 mph, and at sensors in frequently congested stretches; the per-sensor error map shows a cluster on one corridor reaching 7–8 mph. Errors are lowest at night, around 2 mph, when traffic is free-flowing and predictable.

Q60 How do the models behave when a sensor's latest reading is missing? core

Every model gets worse, but differently. At 60 minutes, Graph WaveNet goes from 3.54 to 5.36 mph and gradient boosting from 3.63 to 4.50. The feature model forward-fills the gap and has a missing-now flag, while the deep model sees a raw 0 (-4.57 after scaling). This is the clearest weakness of the deep model, and fixing it (masking or imputing inputs) is future work.

Q61 What did the learned adaptive adjacency learn? tricky

After 25 epochs it is still close to uniform: the average row entropy is 5.23 against a maximum of 5.33. Its top-5 links per sensor overlap with road-graph edges no more than chance (lift 0.87). So most of the spatial benefit here comes from the road-distance supports, and the adaptive matrix acts as a mild global mixing term. Longer training or the ablation run would show whether it sharpens.

Q62 Why are your Graph WaveNet numbers slightly worse than the paper's? deeper

A shorter budget (26 epochs vs 100), batch 32 instead of 64, a single seed, and no learning-rate decay. The gaps are only 0.03–0.06 mph. Our Historical Average (4.19 vs 4.16) and graph statistics (1,722 non-zeros) match the original preprocessing, which validates the pipeline.

Q63 Did you run the LSTM and the Graph WaveNet ablation? tricky

Answer according to the state on viva day. The scripts are ready (`scripts/run_deep_experiments.sh`): the LSTM (time only) and four Graph WaveNet variants with an identical 8-epoch budget (no graph, road graph only, adaptive graph only, both). At the time of writing these runs had not finished because of compute time (about 3 hours on the laptop), so the current results include Graph WaveNet but not these.

Q64 What does the ablation *test*, in principle? deeper

It keeps the architecture and training budget identical and varies only the spatial module. The gap between "no graph" (a pure temporal convolution network) and the graph variants measures the value of spatial information. Comparing "road only" with "adaptive only" shows whether the learned graph can replace the predefined one.

I. Tools, libraries and implementation

Q65 What tools and libraries did you use, and for what? core

Python. NumPy and pandas for arrays, time indexes and tables. h5py to read the legacy HDF5 file. SciPy for the DM test's normal distribution. statsmodels for the ADF test and autocorrelation. scikit-learn for Ridge, HistGradientBoostingRegressor, StandardScaler and permutation importance. PyTorch for the LSTM and Graph WaveNet (MPS backend on the Apple M3). Matplotlib for figures. pytest for unit tests. Streamlit, Altair and pydeck for the web app. LaTeX (Tectonic) for the IEEE paper.

Q66 Why PyTorch? core

It is the standard research framework, the reference Graph WaveNet code is in PyTorch, its eager execution makes debugging easy, and it supports Apple-silicon GPUs through MPS. `torch.einsum` expresses the graph propagation in one line.

Q67 Why scikit-learn's HistGradientBoosting and not XGBoost or LightGBM? deeper

It uses the same histogram algorithm as LightGBM, supports the absolute-error loss and early stopping natively, and needs no extra dependency, so the whole project stays in one ecosystem with permutation importance and pipelines. Performance would be very similar with LightGBM.

Q68 Why did you need h5py to read the data? tricky

The HDF5 file was written by pandas 0.15 (2014), and modern `pd.read_hdf` fails on that format. We read the underlying PyTables arrays (`axis0`, `axis1`, `block0_values`) directly in `load_speed()`, and a unit test checks the shape, column order and timestamps.

Q69 What hardware did you use? core

An Apple M3 laptop with 8 GB of unified memory. Deep models use the GPU via PyTorch MPS (the code picks CUDA, then MPS, then CPU automatically). Gradient boosting runs on the CPU (about 10 minutes for 12 horizons).

Q70 How is the code organised? core

`src/traffic/` is an installable package: data, features, metrics, models, trainer, eda, plotting. `scripts/` has one entry point per experiment. `tests/` has 23 pytest tests. `results/` holds metrics, tables, figures, logs and training histories. `report/` is the IEEE LaTeX paper, `app/` the web app, `notebooks/` the EDA notebook, and `docs/` the literature review and these documents.

Q71 What do your unit tests check? core

No future leakage in features; forward-fill is causal; the weekly profile and scaler use training data only; the split is chronological, disjoint and matches the literature sizes; the adjacency has the expected properties (1,722 non-zeros, threshold); windows align correctly; metrics ignore zero targets; the DM test has the right sign; and every model outputs the right shapes, including a test that the no-graph variant keeps sensors independent.

Q72 How does the trainer build batches so fast? deeper

The full normalised series (34,272 × 207 × 2) is moved to the GPU once. A batch is built by indexing it with anchor offsets (`WindowSampler.batch`), so no Python loop or CPU-to-GPU copy happens per sample.

Q73 How do I reproduce everything from scratch? core

`pip install -e ".[dev]"`, then `python scripts/run_eda.py`, `python scripts/run_classical.py`, `bash scripts/run_deep_experiments.sh`, `python scripts/evaluate.py` and `pytest`. The app runs with `streamlit run app/app.py`.

J. The web application

Q74 Demonstrate and explain your app. core

It is a Streamlit page at localhost:8501. You pick a date, time and sensor and click Run forecast. It shows the current speed, forecasts for +15, +30 and +60 minutes, a chart of the last hour and the forecast against what actually happened, and a map of all 207 sensors coloured by forecast congestion. Buttons step through time, jump to rush hours, play through the day or pick interesting sensors. Two more tabs show the whole day and the test-set accuracy against the baselines.

Q75 Does the app retrain the model? tricky

No. It only does inference. On start-up it loads the dataset and the saved checkpoint once (`st.cache_resource`), and each forecast is a single forward pass on one window (about 6 ms on the laptop GPU, 16 ms on a CPU). The accuracy tab reads the saved test-set metrics, so nothing heavy is recomputed when the page opens.

Q76 Is the app "real time"? tricky

It replays the 2012 test period, so it is a realistic simulation, not a live feed. For live use you would stream current detector data (for example from Caltrans PeMS) into the same 12-step window and call the same model, which is fast enough to update every 5 minutes for the whole network.

Q77 Which libraries does the app use and why? core

Streamlit for the page, widgets and caching, because it turns a Python script into a web app without front-end code. Altair (Vega-Lite) for interactive charts with hover tooltips. pydeck (deck.gl) for the map with clickable sensors on a street basemap.

Q78 How are the map colours chosen? deeper

Each sensor's forecast speed at the chosen horizon is put into one of four bands: free-flow ($\geq$ 55 mph), slowing (40–55), congested (25–40) and severe ($<$ 25). The bands use fixed status colours, and the legend shows each band's name and count, so meaning never depends on colour alone.

Q79 What happens if you pick a time with no data? tricky

The app snaps to the nearest available forecast time and says so in the sidebar (the test period starts at 04:10 on 4 June). If the chosen sensor was offline, it says the forecast cannot be checked and suggests another sensor.

K. Limitations, future work and curveballs

Q80 What are the limitations of your work? core

One city and one season (4 months of 2012). Speed only (no flow, weather or incident data). A static distance graph. A single seed and a shortened Graph WaveNet run. The LSTM and ablation results are pending. The masked metric hides performance during sensor outages. The deep model is sensitive to missing inputs. And the test period has more missing data than training.

Q81 What would you do next? core

Finish the LSTM and ablation runs; train Graph WaveNet for longer with multiple seeds and learning-rate decay; handle missing inputs explicitly (masking channel or imputation); test generalisation on PEMS-BAY; add uncertainty (quantile loss or conformal prediction intervals); try newer models (STAEformer, STID); and connect the app to a live data feed.

Q82 Could this work in another city? tricky

The pipeline, yes: given sensor readings and road distances, everything rebuilds automatically. The trained weights, no: the adaptive embeddings, sensor statistics and weekly profiles are specific to these 207 sensors. A new network needs retraining, and transfer learning across cities is an open research topic.

Q83 If a new sensor is added to the network, what breaks? tricky

Graph WaveNet has a fixed node count (207) and per-node embeddings, so it must be retrained or extended. The feature-based gradient boosting model is easier: a new sensor needs some history to compute its weekly profile and statistics, after which the same trees apply, because they are shared across sensors.

Q84 Is 3.07 mph good? tricky

In context, yes. It is 27% better than the seasonal average, matches the published Graph WaveNet within a few hundredths of a mph, and beats DCRNN. For perspective, typical freeway speed is about 60 mph, so it is about a 5% average error. The best recent models (e.g. STAEformer, 3.34 at 60 minutes) are only modestly better.

Q85 What is the most surprising thing you found? tricky

Two things. How strong a well-featured gradient boosting model is: within 0.07 mph of Graph WaveNet at 60 minutes. And that the deep model's advantage disappears, and reverses, when the latest input is missing. Both suggest that data handling matters as much as architecture.

Q86 If you had to deploy one model tomorrow, which and why? tricky

Graph WaveNet for accuracy, with a fallback to gradient boosting (or forward-filled inputs) for sensors whose latest reading is missing, since that is where the deep model is weakest. Both are fast enough to forecast the whole network every 5 minutes.
